Gestion de la mémoire

La mémoire principale est le lieu où se trouvent les programmes et les données quand le processeur les exécute.

D'où la nécessité de la gérer de manière optimale, car en dépit de sa grande disponibilité, elle n'est, en général, jamais suffisante.

Ainsi il faut :

Optimiser l ’utilisation de la mémoire principale = RAM

- connaître et allouer les parties libres de la RAM
- récupérer la mémoire libérée par la terminaison d’un processus

Offrir aux processus les services de la mémoire virtuelle:
Taille supérieure à celle de la RAM disponible au moyen des techniques de va et vient et pagination

Mémoire/Adresses physiques et logiques

- **Mémoire physique:** la mémoire RAM de la machine
- **Adresses physiques:** les adresses de cette mémoire

- **Mémoire logique:** l’espace d’adressage d’un programme

une adresse logique est une adresse à une location de programme

- par rapport au programme lui-même seulement
- indépendante de la position du programme en mémoire physique

- Par exemple le 50ème mot depuis le début d'espace mémoire logique

Adresses

Adresse d'une information :

Chaque information est repérée par son adresse, c'est à dire le numéro de son premier

octet.

Cet adressage en termes d'octets permet d'adresser, de repérer, en mémoire tous types d'information, quelque soit leur taille.

On peut ainsi adresser un simple octet

(un char en langage C), un mot sur deux octets (un short en langage C), etc.

Translation d'adresse logique en adresse physique

- Les adresses logiques sont traduites en adresses physiques après chargement du programme en mémoire central RAM par le **MMU**

(MMU = memory management unit = unité de gestion de mémoire= unité de traduction adresses)

Mémoire virtuelle

La mémoire virtuelle est une fonctionnalité d'un SE qui permet d'augmenter artificiellement la mémoire vive (RAM) d'un ordinateur, en transférant temporairement des données vers le disque dur de l'ordinateur.

Les données transférées doivent être ramener en RAM en cas de besoin.

Pourquoi une mémoire virtuelle ?

Mémoire **physique** coûteuse. Mémoire **secondaire** (disques, mémoire étendue, ...) peu coûteuse.

Programmes gourmands en mémoire et qui ne "tiennent pas" toujours en RAM.

Utiliser la mémoire secondaire "comme" mémoire RAM.

Avec le recours à la mémoire virtuelle, un processus peut être chargé dans ***des pages*** **non contiguës.**

La pagination

- La mémoire logique et la mémoire physique sont structurées en unités d'allocations

  (**pages** pour la mémoire logique

  et

  cases ou **cadres** pour la mémoire physique).

- La taille d’une page est fixe et égale à celle d’une case. Elle varie (en générale) entre 512 octets et 8 Ko.

Le format de l’adresse logique est :
(numéro de page, déplacement dans la page)

- Il peut y avoir une fragmentation interne si la dernière page de l’espace d’adressage logique n’est pas pleine.

Table des pages

- La correspondance entre les pages et les cases est mémorisée dans une table appelée **table des pages**.

- Le nombre d’entrées dans la table est égal au nombre de pages logique.

- La table des pages d’un processus doit être (en totalité ou en partie) en mémoire centrale lors de l’exécution du processus.

- Elle est nécessaire pour la conversion des adresses virtuelles en adresses physiques.

- Chaque entrée de la table des pages est composée de plusieurs champs, notamment :

- Les bits de protection, définissent le niveau de protection de la page, en particulier si elle est partagée et quels sont ses droits d’accès

- Le bit de présence, positionné si la page est présente en MC

- Le bit de référence, positionné à chaque accès à la page et remis à 0 à chaque interruption d'horloge

- Le bit de modification (M), indique que la page a été modifiée par rapport à sa version sur disque

- Le numéro de case dans la MC correspondant à la page

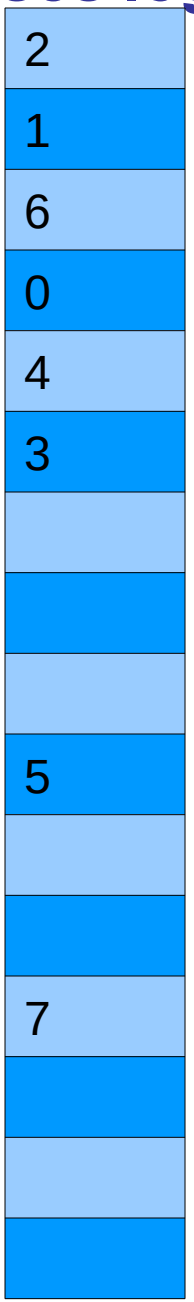
Conversion d'adresses logiques

- La relation entre les adresses logique et physiques est indiquée dans la **table des pages**

- Dans l'exemple, un ordinateur peut produire des adresses sur 16 bits (64 Ko) mais il n'y a que 32 Ko de mémoire physique.

- La mémoire logique est divisée en pages de 4K

- La mémoire physique est divisée en **cadre de 4k** (*page frame*)



L’adresse logique

20500=5*4096+20

est transformée en adresse physique

12308=3*4096+20

Fonctionnement interne d'une MMU avec 16 pages de 4Ko et une RAM de 32 ko

La MMU reçoit, en entrée une adresse logique et envoie en sortie **l'adresse physique** ou provoque un **déroutement**.

Dans la figure précédente, nous avons 16 pages virtuelles de 4096 octets chacune → les adresses virtuelles varient de 0 à

$16x4096-1=2^4x2^{12}-1 =2^{16}$ -1→ l'adressage se fait sur 16 bits

4 bits pour le numéro de la page → 2^4 = 16 pages

12 bits pour adresser l'ensemble des 4096 octets d'une page

- Pour les adresses physiques: nous avons **8** pages de 4096 octets chacune : soit **$2^3 \times 2^{12} = 2^{15}$ adresses** → l'adressage se fait sur 15 bits.

L’adresse logique sur 16 bits qui arrive au MMU est divisée en deux parties: un numéro de page sur **4** bits et un décalage sur **12** bits

- La MMU examine l'entrée dans la **Table de pages** correspondant au numéro de page,

- Si le bit de présence est à 0, la page n'est pas en mémoire MC alors le MMU provoque un déroutement et le processeur est rendu au SE.

- Le SE charge la page demandée

- Si le bit de présence est à 1, **Rappelons l'adresse physique est sur 15 bits**

- il détermine l'adresse physique en recopiant dans les 3 bits de poids le plus fort le numéro de case correspondant au numéro de page virtuel (3 bits car le nombre de pages physiques 8=$\mathbf{2^3}$) auxquels sont ajoutés les 12 bits de décalage, qui sont copiés à partir de l'adresse logique sans être modifiés

→ Ils forment ensemble une adresse physique sur 15 bits

- L'adresse virtuelle **8196=**0010 0000 00000100 (en binaire)

Donc **page Virtuelle = 2=010** chargée dans la case 6=110

Donc l'adresse physique = 110 0000 00000100 **= 24580**

Algorithmes de remplacement de page

- A la suite d'un défaut de page, le système d'exploitation doit ramener en mémoire la page manquante à partir du disque.

- S'il n'y a pas de cadres libres en mémoire, il doit retirer une page de la mémoire pour la remplacer par celle demandée.

- Si la page à retirer a été modifiée depuis son chargement en mémoire, il faut la réécrire sur le disque.

- Quelle est la page à retirer de manière à minimiser le nombre de défauts de page ?

Algorithme de remplacement optimal (Algorithme de Belady)

- Cet algorithme et le meilleur possible, car il minimise le nombre de défauts de page
- Le SE indexe chaque page par le nombre d'instructions qui seront exécutées avant qu'elle ne soit référencée.
- En cas de nécessité, le SE retire la page d'indice le plus élevé, c'est à dire la page qui sera référencée dans le futur le plus lointain.
- Cet algorithme est pratiquement impossible à appliquer (comment calculer les indices des pages ?).
- Toutefois, avec un simulateur, on peut évaluer les performances de cet algorithme et s'en servir comme référence pour les suivants

Exemple: Soit un système avec 3 cases de RAM et une suite de références w = {**7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1**}.

La suite de références est constituée de la liste des numéros des pages virtuelles accédées dans le temps,

Représentation de la RAM

Chaque élément de ligne **i** et colonne **j** montre la page chargée dans la case **i** après avoir été référencée.

Remplacement de la page non récemment utilisée (NRU, Not Recently Used)

- Cet algorithme utilise les bits **R** et **M** associés à chaque page pour déterminer les pages non récemment utilisées.

- Le bit **R** est positionné à **1** chaque fois qu'une page est référencée (accédée) . Le bit **R** est remis à **0** à chaque interruption d'horloge

- Le bit **M** est positionné lorsque la page est modifiée (elle n'est plus identique à sa copie sur disque).

- Lorsqu'un défaut de page se produit, l'algorithme NRU sélectionne la page à retirer en procédant comme suit :

  - Il vérifie s'il existe des pages non référencées et non modifiées (**R=0** et **M=0**). Si c'est le cas, il sélectionne une page et la retire.

  - Sinon, il vérifie s'il existe des pages non référencées et modifiées (**R=0** et **M=1**). Si c'est le cas, il sélectionne une page et la retire (une sauvegarde sur disque de la page retirée est nécessaire).

  - Sinon, il vérifie s'il existe des pages référencées et non modifiées (**R=1** et **M=0**). Si c'est le cas, il sélectionne une page et la retire.

  - Sinon, il sélectionne une page référencée et modifiée et la retire (**R=1** et **M=1**). Dans ce cas, une sauvegarde sur disque de la page retirée est nécessaire.

On peut organiser l’ensemble des pages ainsi:

- Classe 0: Non référencée, non modifiée
- Classe 1: Non référencée, modifiée
- Classe 2: Référencée, non modifiée
- Classe 3: Référencée, modifiée

L’algorithme NRU enlève une page au hasard dans la classe la plus basse non vide

Remplacement de page FIFO

- Il mémorise dans une file FIFO les pages présentes en mémoire.

- Lorsqu'un défaut de page se produit, il retire la plus ancienne, c'est à dire celle qui se trouve en tête de file.

- **Ce n’est pas une bonne stratégie :** Son critère n’est pas fondé sur l’utilisation de la page

- **Anomalie de Belady (contre exemple de belady) pour FIFO**

- Intuitivement, on peut imaginer que plus il existe de cadres, moins il y’a de défauts de pages.

- On réalité, on peut rencontrer des exemples où en augmentant le nombre de cadres on augmente le nombre de défauts de page au lieu de le diminuer.

Anomalie de Belady:

Exemple avec 3 cadres

7	0	1	2	7	0	3	7	0	1	2	3	7	0	1	2	3	2	4	3
7	7	7	2	2	2	3	3	3	3	3	3	7	7	7	2	2	2	2	2
	0	0	0	7	7	7	7	7	1	1	1	1	0	0	0	3	3	3	3
		1	1	1	0	0	0	0	0	2	2	2	2	1	1	1	1	4	4

Nombre d'accès: 20
Fautes de page: 15
(75%)

Exemple avec 4 cadres

Nombre d'accès: 20
Fautes de page: 16
(80%)

Remplacement de la page la moins récemment utilisée (LRU = Least Recently Used)

- Remplace la page dont la dernière référence remonte au temps le plus lointain (**le passé utilisé pour prédire le futur)**
- En raison de la localité des références, il s'agit de la page qui a le moins de chance d'être référencée

Exemple LRU avec 3 cases

7012 0304 2303 2120 1701

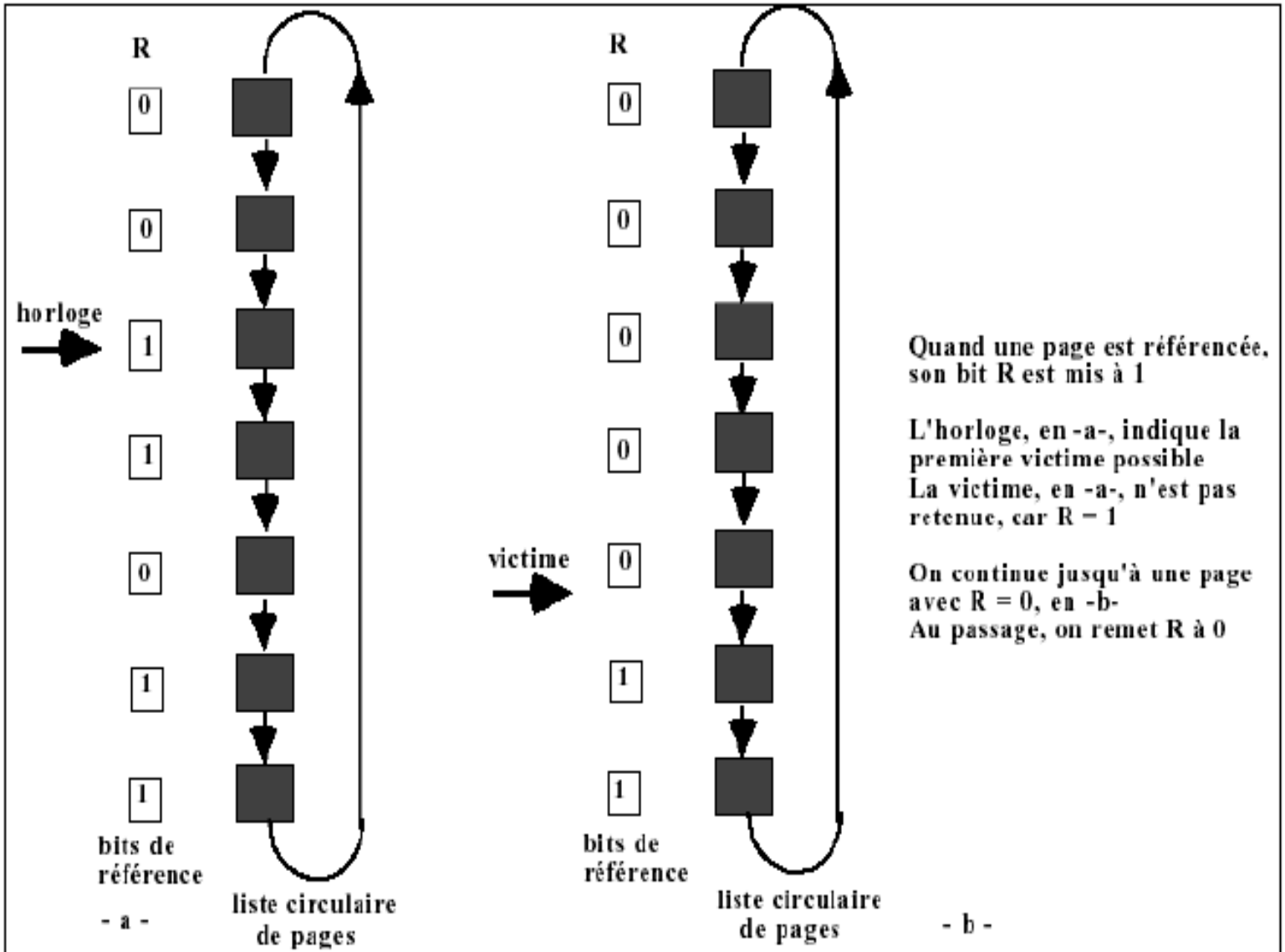
Nombre d'accès: 20
Fautes de page: 12
(60%)

Même exemple avec FIFO

Nombre d'accès: 20
Fautes de page: 15
(75%)

Algorithme de l’horloge (seconde chance)

- Semblable à FIFO, mais les cadres qui viennent d’être utilisés (bit=1) ne sont pas touchés (deuxième chance)
- les pages en mémoire sont mémorisées dans une liste circulaire en forme d'horloge
- Un indicateur pointe sur la page la plus ancienne
- Lorsqu’un défaut de page se produit, les pages sont examinées, une par une, en commençant par celle pointée par l’indicateur.
- La première page rencontrée ayant son bit de référence R à 0 est remplacée. Le bit R de la page ajoutée est à 1.
- Si le bit R d’une page examinée est 1, il est mis à 0.



ALGORITHME DE LA SECONDE CHANCE (ALGORITHME DE L'HORLOGE)